

LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS
IN PRACTICE**

8 - Pandas and Seaborn

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

PANDAS

Introduction to Pandas

Pandas is a powerful Python library for data manipulation and analysis. It is used for storage, manipulation, and visualization of multidimensional data. It is more flexible than Ndarrays and also compatible with it. It means that we can use Ndarrays to create Pandas data structures.

Key features:

- DataFrame object for data manipulation with integrated indexing
- Tools for reading and writing data between different formats
- Data alignment and handling of missing data
- Reshaping and pivoting of datasets
- Data filtering, selection, and transformation
- Time-series functionality

3

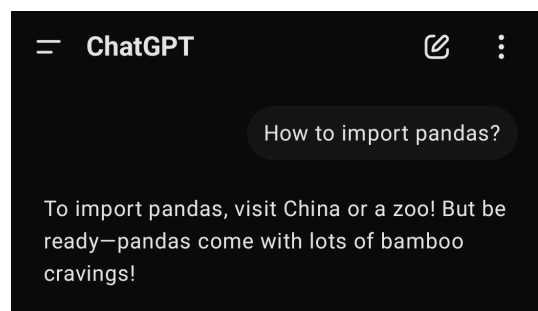
Installing Pandas

Pandas can be installed using pip:

```
pip install pandas
```

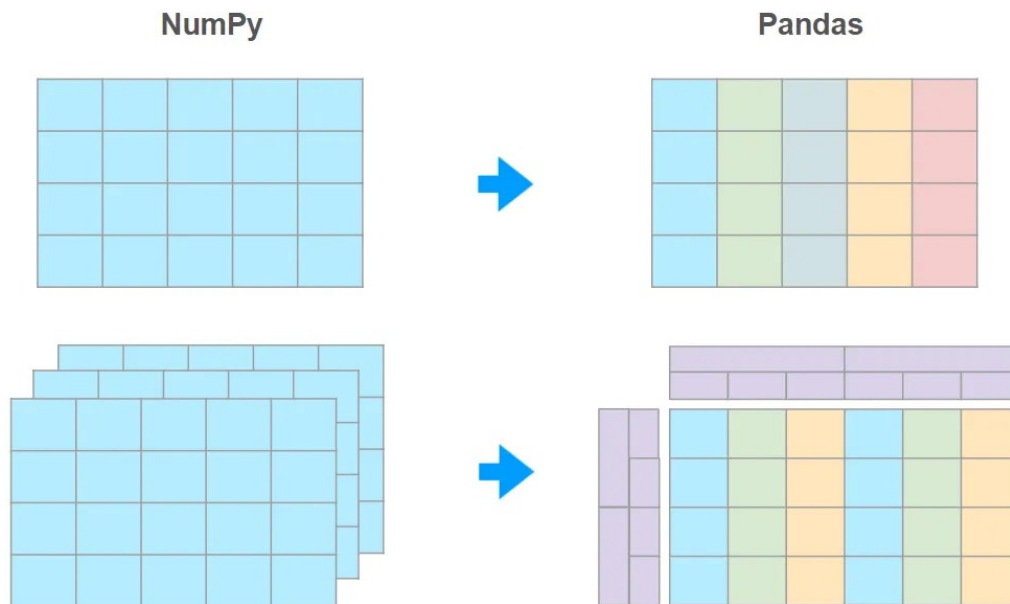
Basic import convention:

```
import pandas as pd
```



4

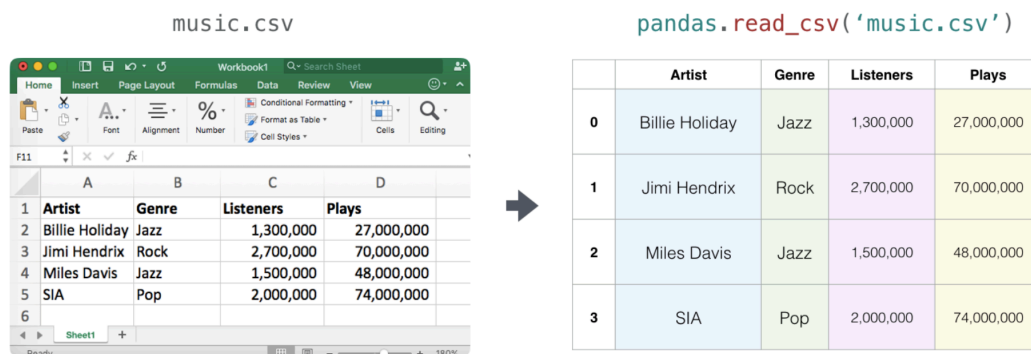
Pandas Data Structures



5

Credits Lev Maximov

Pandas Data Structures



Two key features that Pandas brings to NumPy arrays are:

1. **Heterogeneous types** — each column is allowed to have its own type;
2. **Index** — improves lookup speed for the specified column(s).

It turns out these features are enough to make Pandas a powerful competitor to both spreadsheets and databases.

6

Pandas Data Structures

Two primary data structures in Pandas:

- **Series:** one-dimensional labeled array
 - Similar to a column in a spreadsheet
 - Contains a sequence of values and an associated array of labels (index)
- **DataFrame:** two-dimensional labeled data structure
 - Similar to a spreadsheet or SQL table
 - Collection of Series objects that share an index
 - Most commonly used Pandas object

7

Creating Series

A Pandas series is a homogeneous one-dimensional array with an index. It can store the data of any supported type. We can use lists or Ndarrays to create series in Pandas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

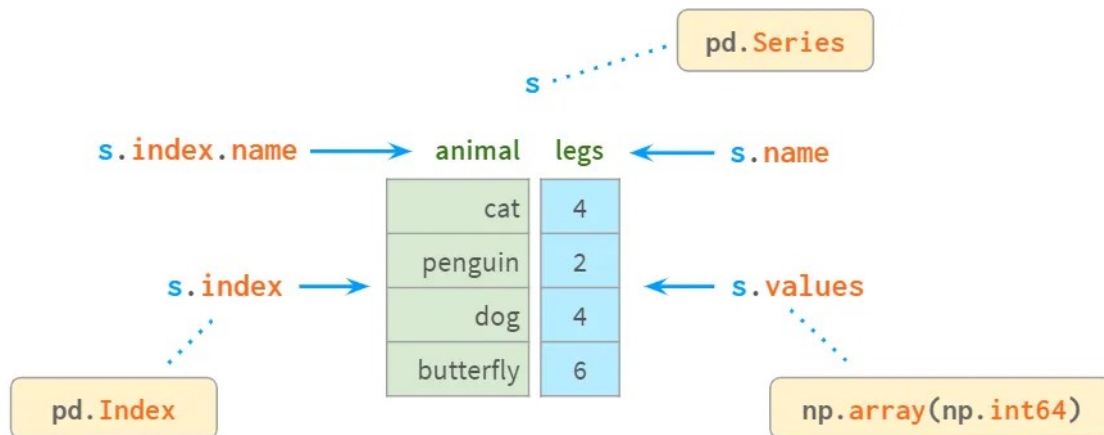
s1 = pd.Series([1, 2, 3, 4, 5])
print(type(s1)) # pandas.core.series.Series

s2 = pd.Series(np.arange(5), dtype=np.uint8)
print(s2)
# outputs:
# 0 0
# 1 1
# 2 2
# 3 3
# 4 4
# dtype: uint8
```

8

Series

Internally, a Series store the *values* into a Numpy array. But it adds an *index* to access values by *labels*.



9
Credits Lev Maximov

Series' properties

Series have the following interesting properties:

```
s1 = pd.Series([1, 2, 3, 4, 5])
```

```
s1.values  
# array([1, 2, 3, 4, 5])
```

```
s1.array  
# <NumpyExtensionArray>  
# [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]  
# Length: 5, dtype: int64
```

```
s1.index  
# RangeIndex(start=0, stop=5, step=1)
```

```
s1.dtype  
# dtype('int64')
```

```
s1.shape  
# (5,)
```

10

Series

Labels do not need to be unique, but uniqueness is required to boost the lookup speed and is assumed in many operations.

```
pd.Series([0.6, 0.7, 2.2, 10.5],  
          index=['Athens', 'Oslo', 'Paris', 'Bangkok'])
```



s		
Athens	0.6	0
Oslo	0.7	1
Paris	2.2	2
Bangkok	10.5	3

```
s.dtype == np.float64
```

11

Credits Lev Maximov

Series data selection

Every element can be addressed in two alternative ways, by **label** i.e. *an index* and by **position** i.e. *directly without the index*

s		
Athens	0.6	0
Oslo	0.7	1
Paris	2.2	2
Bangkok	10.5	3

s.Paris s['Paris']
0.6
0.7
2.2
10.5

s[2:3]

s['Oslo': 'Paris']
0.6
0.7
2.2
10.5

s[1:3]

s[':Oslo']
0.6
0.7
2.2
10.5

s[:2]

12

Credits Lev Maximov

Series data selection

However, what if labels are of type integers, similarly as the positions?

Pandas provides properties to distinguish between the two cases:

	s		s.loc['Paris']	s.loc['Oslo':'Paris']	s.loc[:'Oslo']
Athens	0.6	0	0.6	0.6	0.6
Oslo	0.7	1	0.7	0.7	0.7
Paris	2.2	2	2.2	2.2	2.2
Bangkok	10.5	3	10.5	10.5	10.5

s.iloc[2] s.iloc[1:3] s.iloc[:2]

13

Credits Lev Maximov

Creating DataFrames

DataFrames are like tables in a database. They can be created from various data sources:

```
# From a dictionary of lists
data = {'Name': ['John', 'Anna', 'Peter', 'Linda'],
        'Age': [28, 34, 29, 42],
        'City': ['New York', 'Paris', 'Berlin', 'London']}
df = pd.DataFrame(data)

print(df)
#      Name  Age   City
# 0   John   28 New York
# 1   Anna   34   Paris
# 2  Peter   29   Berlin
# 3  Linda   42   London
```

14

Creating DataFrames

DataFrames are like tables in a database. They can be created from various data sources:

```
# From a list of dictionaries
data = [{'Name': 'John', 'Age': 28},
        {'Name': 'Anna', 'Age': 34}]
df = pd.DataFrame(data)
```

```
print(df)
#    Name  Age
# 0  John   28
# 1  Anna   34
```

```
# From a CSV file
df = pd.read_csv('data.csv')
```

15

Creating Dataframes

Columns order can be specified at creation:

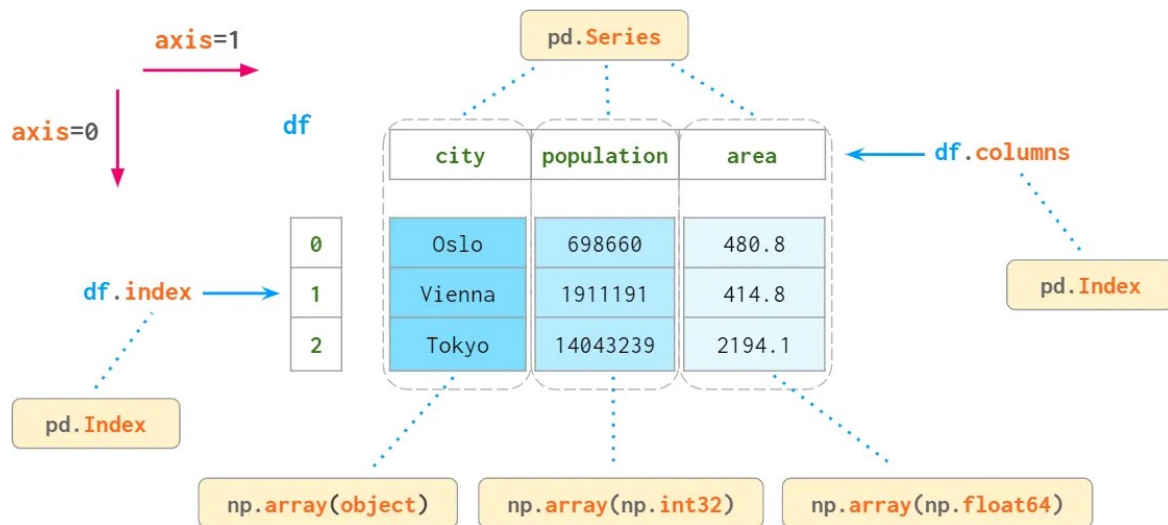
```
data = {'city': ['Bangalore', 'Bangalore', 'Bangalore',
                'Mumbai', 'Mumbai', 'Mumbai'],
        'year': [2020, 2021, 2022, 2020, 2021, 2022,],
        'population': [10.0, 10.1, 10.2, 5.2, 5.3, 5.5]}

# we can specify the columns order
df2 = pd.DataFrame(data, columns=['year', 'city', 'population'])
```

```
print(df2)
# outputs:
# year city population
# 0 2020 Bangalore 10.0
# 1 2021 Bangalore 10.1
# 2 2022 Bangalore 10.2
# 3 2020 Mumbai 5.2
# 4 2021 Mumbai 5.3
# 5 2022 Mumbai 5.5
```

16

Creating Dataframes



17

Data Import and Export

Pandas supports multiple file formats:

```
# Reading data
df = pd.read_csv('file.csv')      # CSV files
df = pd.read_excel('file.xlsx')   # Excel files
df = pd.read_json('file.json')    # JSON files
df = pd.read_sql('query', connection) # SQL databases

# Writing data
df.to_csv('output.csv')           # CSV files
df.to_excel('output.xlsx')        # Excel files
df.to_json('output.json')         # JSON files
df.to_sql('table_name', connection) # SQL databases
```

18

Data Exploration

Essential methods for initial data exploration:

```
df.head()      # First 5 rows
df.tail()      # Last 5 rows
df.info()      # DataFrame information
df.describe()  # Statistical summary
df.shape       # Dimensions (rows, columns)
df.columns     # Column names
df.dtypes      # Data types of each column
df.isnull().sum() # Count of missing values
```

19

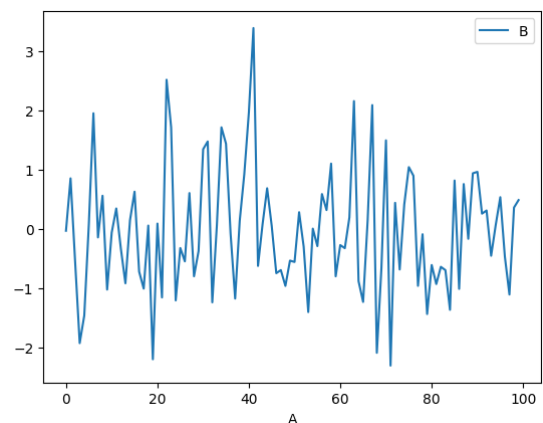
Data Visualization

Objects of data structures can directly call Matplotlib visualization functions like plot(). Basically, Pandas provides a wrapper for several of these functions.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df1 = pd.DataFrame()
df1['A'] = pd.Series(list(range(100)))
df1['B'] = np.random.randn(100, 1)

# plot it
df1.plot(x='A', y='B')
plt.show() # not needed in jupyter
```



20

Data Visualization

Pandas integrates with matplotlib for easy plotting:

```
# Basic plots
df['Age'].plot()           # Line plot
df['Age'].plot.hist()      # Histogram
df.plot.scatter(x='Age', y='Salary') # Scatter plot
df.plot.bar()             # Bar plot
df.plot.box()             # Box plot

# Multiple plots
df.plot.line(subplots=True, figsize=(10, 8))

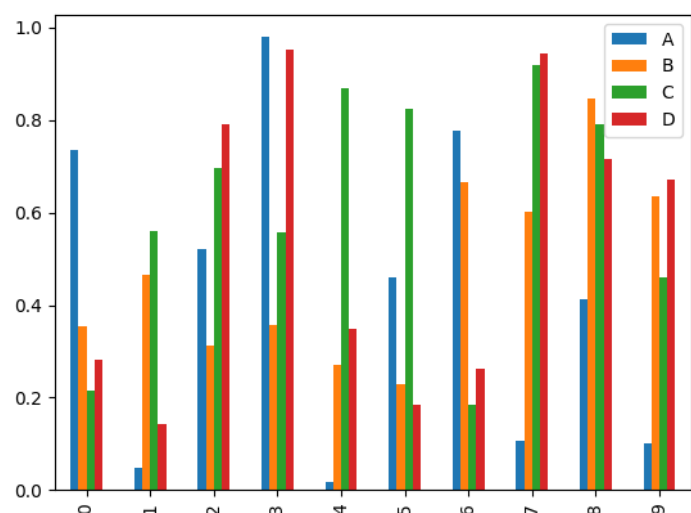
# Customizing plots
df.plot.pie(y='Amount', figsize=(8, 8), autopct='%1.1f%%')
```

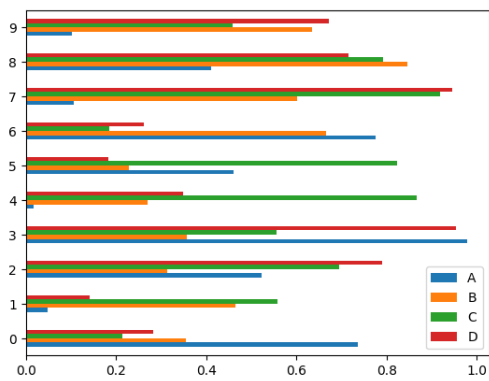
21

Data Visualization

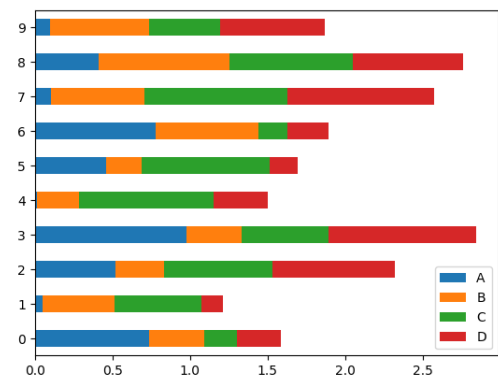
For instance, we can plot a bar graph out of 4 series:

```
df2 = pd.DataFrame(np.random.rand(10, 4),
                    columns=['A', 'B', 'C', 'D'])
df2.plot.bar()
```

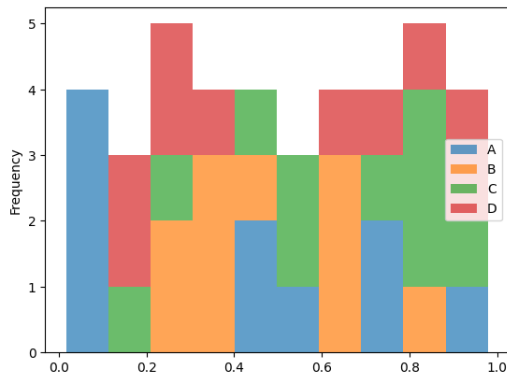




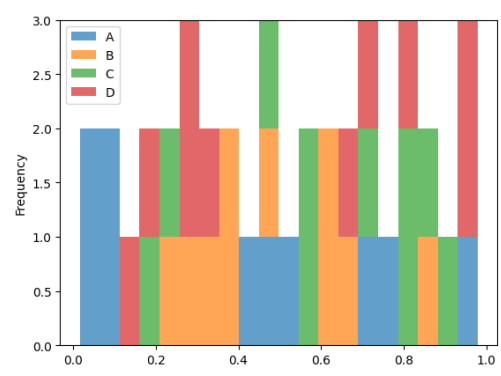
df2.plot.barh()



df2.plot.barh(stacked = True)

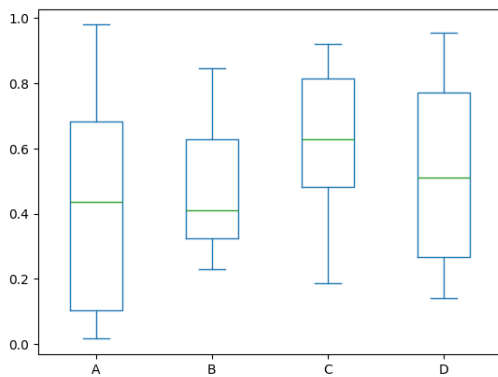


df2.plot.hist(stacked=True, alpha=0.7)

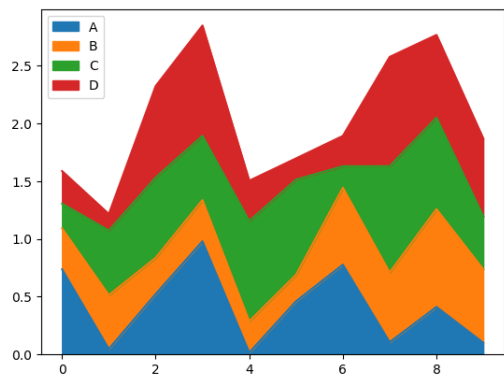


df2.plot.hist(stacked=True, alpha=0.7, bins=20)

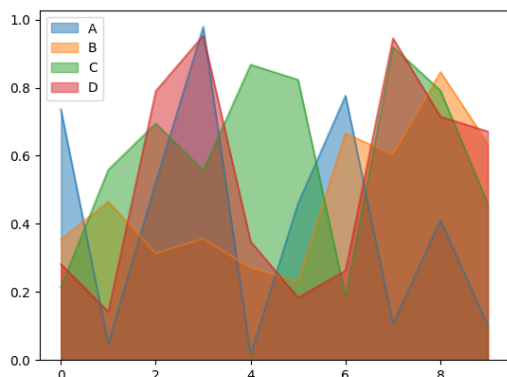
23



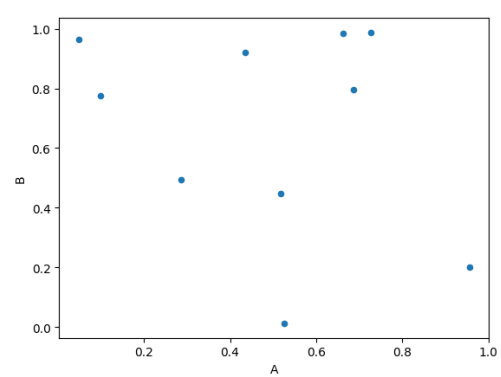
df2.plot.box()



df2.plot.area()



df2.plot.area(stacked=False)



df2.plot.scatter('A', 'B')

24

Data Selection

Data can be selected, similarly as slicing in Numpy. Labels for columns, numbers for rows. Both using *loc* property.

df				
	Artist	Genre	Listeners	Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000
1	Jimi Hendrix	Rock	2,700,000	70,000,000
2	Miles Davis	Jazz	1,500,000	48,000,000
3	SIA	Pop	2,000,000	74,000,000

`df['Artists']`

	Artist
0	Billie Holiday
1	Jimi Hendrix
2	Miles Davis
3	SIA

`df[1:3]`

	Artist	Genre	Listeners	Plays
1	Jimi Hendrix	Rock	2,700,000	70,000,000
2	Miles Davis	Jazz	1,500,000	48,000,000

`df.loc[1:3, ['Artist']]`

	Artist
1	Jimi Hendrix
2	Miles Davis
3	SIA

25

Indexing

In pandas, *.loc* and *.iloc* properties are used for selecting data from a DataFrame, but they differ in how they reference rows and columns:

- **.loc[] (Label-Based Indexing)**

- Selects data by explicit labels (index and column names).
- Syntax: `df.loc[row_label, column_label]`
- Includes both start and end labels in slices.

```
df = pd.DataFrame({'A': [10, 20, 30],
                   'B': [40, 50, 60]},
                  index=['x', 'y', 'z'])
```

```
# Select row 'y' and column 'A'
print(df.loc['y', 'A']) # Output: 20
```

```
# Select multiple rows and columns
print(df.loc[['x', 'z'], ['A', 'B']])
```

26

Indexing

- **.iloc[] (Integer-Based Indexing)**

- Selects data by position (row and column numbers).
- Syntax: `df.iloc[row_index, column_index]`
- Zero-based, end index is excluded in slices.

```
df = pd.DataFrame({'A': [10, 20, 30],  
                  'B': [40, 50, 60]},  
                  index=['x', 'y', 'z'])
```

```
# Select second row (index 1) and first column (index 0)  
print(df.iloc[1, 0]) # Output: 20
```

```
# Select multiple rows and columns by position  
print(df.iloc[[0, 2], [0, 1]])
```

27

Data Filtering

But slicing is only the basics. Data can also be filtered based on conditions:

```
# Simple conditions
```

```
df[df['Age'] > 30] # Age greater than 30  
df[df['City'] == 'New York'] # City is New York
```

```
# Multiple conditions
```

```
df[(df['Age'] > 30) & (df['City'] == 'New York')] # AND  
df[(df['Age'] > 30) | (df['City'] == 'New York')] # OR
```

```
# Other filters
```

```
df[df['Name'].isin(['John', 'Anna'])] # Values in a list  
df[df['City'].str.contains('New')] # String contains
```

28

Handling Missing Values

Many datasets you'll deal with will have missing values. Pandas can handle them in various ways:

`df`

	Artist	Genre	Listeners	Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000
1	Jimi Hendrix	Rock	2,700,000	NaN
2	Miles Davis	Jazz	1,500,000	48,000,000
3	SIA	Pop	2,000,000	74,000,000

`df.dropna()`

	Artist	Genre	Listeners	Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000
2	Miles Davis	Jazz	1,500,000	48,000,000
3	SIA	Pop	2,000,000	74,000,000

29

Handling Missing Values

Pandas can detect them, remove rows or columns, fill them with default or derived values:

```
# Detecting missing values
df.isnull()          # Return boolean DataFrame
df.isnull().sum()    # Count missing values per column

# Handling missing values
df.dropna()          # Drop rows with any missing values
df.dropna(axis=1)    # Drop columns with any missing values
df.fillna(0)         # Replace missing values with 0
df.fillna(method='ffill') # Forward fill (use previous value)
df.fillna(method='bfill') # Backward fill (use next value)
df['Age'].fillna(df['Age'].mean()) # Fill with mean
```

30

Grouping and Aggregating

Things start to get really interesting when you start grouping rows with certain criteria and aggregating their data.

df					df.groupby('Genre').sum()		
	Artist	Genre	Listeners	Plays		Listeners	Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000	Genre		
1	Jimi Hendrix	Rock	2,700,000	70,000,000	Jazz	2,800,000	75,000,000
2	Miles Davis	Jazz	1,500,000	48,000,000	Pop	2,000,000	74,000,000
3	SIA	Pop	2,000,000	74,000,000	Rock	2,700,000	70,000,000

31

Grouping and Aggregating

Powerful operations for summarizing data:

```
# Group by one column
df.groupby('City').mean()      # Mean of numeric columns per city
df.groupby('City').count()     # Count of values per city
df.groupby('City').size()      # Size of each group

# Group by multiple columns
df.groupby(['City', 'Gender']).mean()

# Custom aggregations
df.groupby('City').agg({'Age': 'mean', 'Salary': ['min', 'max']})
df.groupby('City').agg({'Age': ['min', 'max', 'mean', 'count']})
```

32

Data Transformation

Common operations for transforming data:

```
# Adding new columns
df['DoubleAge'] = df['Age'] * 2
df['FullName'] = df['FirstName'] + ' ' + df['LastName']

# Applying functions
df['Age'].apply(lambda x: x * 2) # Using lambda
df['Name'].apply(len)           # Getting string lengths

# Transforming values
df['City'] = df['City'].str.upper() # Convert to uppercase
df['Code'] = df['Code'].astype(str) # Change data type
```

33

Creating New Columns

We can create new columns based on the data already available in the other columns:

```
df['Avg Plays'] = df['Plays']/df['Listeners']
```

	Artist	Genre	Listeners	Plays	Avg Plays
0	Billie Holiday	Jazz	1,300,000	27,000,000	20
1	Jimi Hendrix	Rock	2,700,000	70,000,000	25
2	Miles Davis	Jazz	1,500,000	48,000,000	32
3	SIA	Pop	2,000,000	74,000,000	37

34

Arithmetic operations

Ordinary operations like add, subtract, etc. can be applied to dataframes, series and a combination of them.

All arithmetic operations are aligned against the row and column labels:

	A	B	C			B	C	D			A	B	C	D
a	1	2	3		b	1	2	3		a	NaN	NaN	NaN	NaN
b	4	5	6	+	c	4	5	6	=	b	NaN	6.	8.	NaN
c	7	8	9		d	7	8	9		c	NaN	12.	14.	NaN
										d	NaN	NaN	NaN	NaN

35

Arithmetic operations

In mixed operations between DataFrames and Series, the Series is treated like a row vector and aligned accordingly

	A	B	C		B	C	D		A	B	C	D
a	1	2	3					a	NaN	2.	1.5	NaN
b	4	5	6	/	1	2	3	b	NaN	5.	3.	NaN
c	7	8	9		pd.Series			c	NaN	8.	4.5	NaN

36

Arithmetic operations

For Numpy or list, they are broadcasted:

	A	B	C						A	B	C			
a	1	2	3	/	<table><tr><td>1</td><td>2</td><td>3</td></tr></table>	1	2	3	=		a	1.	1.	1.
1	2	3												
b	4	5	6			b	4.	2.5		2.				
c	7	8	9	c	7.	4.	3.							

list or
np.array

To obtain the same result but with a Series, use methods instead of operators:

	A	B	C						A	B	C
a	1	2	3	.div(b , axis=0) =	a	1	pd.Series	a	1.	2.	3.
b	4	5	6		b	2		b	2.	2.5	3.
c	7	8	9		c	3		c	2.3	2.6	3.

37

Merging and Joining

Combining datasets in various ways:

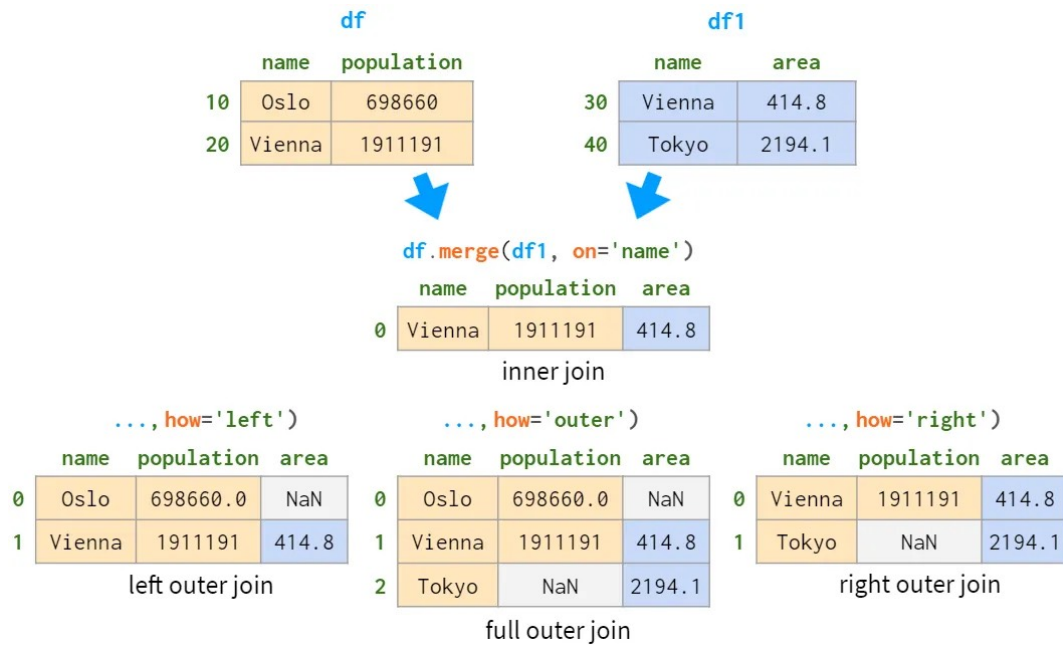
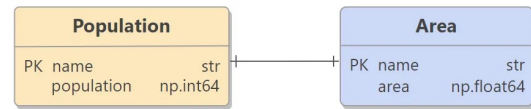
```
# Merge DataFrames (SQL-like join)
pd.merge(df1, df2, on='ID')           # Inner join
pd.merge(df1, df2, on='ID', how='left') # Left join
pd.merge(df1, df2, on='ID', how='right') # Right join
pd.merge(df1, df2, on='ID', how='outer') # Outer join

# Join using indexes
df1.join(df2)                         # Join on index

# Concatenate DataFrames
pd.concat([df1, df2])                 # Stack vertically
pd.concat([df1, df2], axis=1)         # Stack horizontally
```

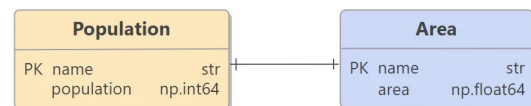
38

Merging and Joining

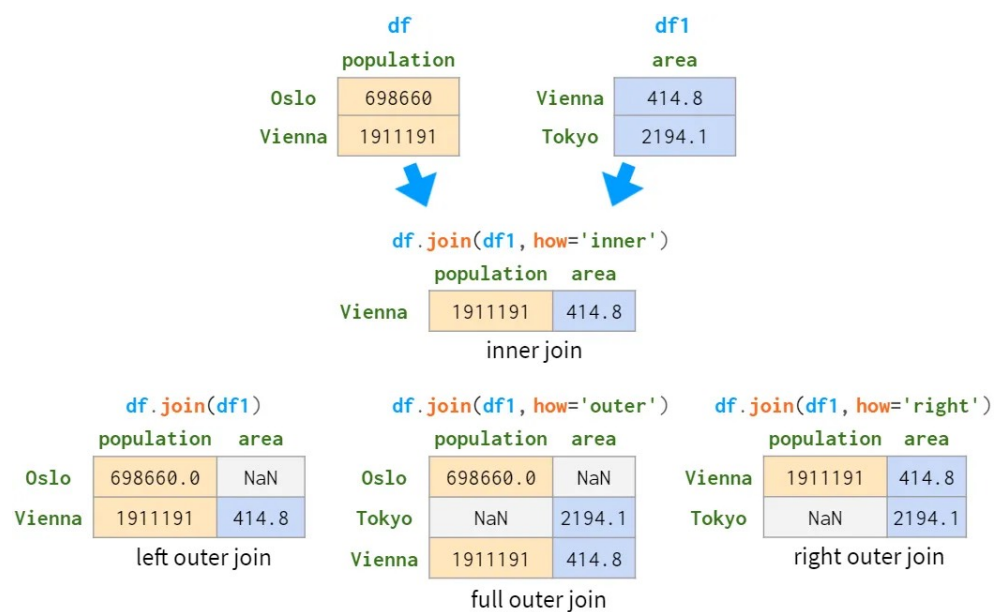


39

Merging and Joining



If the column is already in the index, you can use join:



40

Data Reshaping

Techniques for restructuring data:

```
# Pivot tables
df.pivot_table(index='City', columns='Gender', values='Salary', aggfunc='mean')

# Melt (unpivot) - convert from wide to long format
pd.melt(df, id_vars=['ID'], value_vars=['Score1', 'Score2'],
        var_name='Exam', value_name='Score')

# Stack and unstack
df.stack()    # Pivot from columns to index
df.unstack()  # Pivot from index to columns
```

41

Data Reshaping

df

	client	product	quantity	price
0	John	bananas	5	1.5
1	John	oranges	3	3.
2	Silvia	bananas	4	2.5
3	Silvia	oranges	2	4.



```
df.pivot(index='client', columns='product', values='quantity')
```

client \ product	bananas	oranges
John	5	3
Silvia	4	2

42

Time Series Analysis

Pandas has specialized functionality for time series data:

```
# Creating date ranges
dates = pd.date_range(start='2023-01-01', end='2023-01-10')
dates = pd.date_range(start='2023-01-01', periods=10, freq='D')

# Time-based indexing
df['Date'] = pd.to_datetime(df['Date']) # Convert to datetime
df.set_index('Date', inplace=True)      # Set date as index
df['2023-01']                           # Select by month
df['2023-01-01':'2023-01-05']          # Select date range

# Time-based resampling
df.resample('M').mean()                 # Monthly average
df.resample('W').sum()                  # Weekly sum
```

43

Pandas vs Numpy

Consider the following table, describing the diverse product line of an online shop with a total of four distinct products:

product id	price, \$	weight, g
1	10	100
2	20	150
3	15	200
4	18	80

Pandas vs Numpy

Sorting can be more readable in Pandas:

id	price	weight
1	10	100
2	20	150
3	15	200
4	15	80



id	price	weight
1	10	100
3	15	200
4	15	80
2	20	150

numpy array

```
a[np.argsort(a[:,1])]
```

pandas dataframe

```
df.sort_values('price')
```

Pandas vs Numpy

Even more when multiple columns are involved:

id	price	weight
1	10	100
2	20	150
3	15	200
4	15	80



id	price	weight
1	10	100
4	15	80
3	15	200
2	20	150

numpy array

```
b = a[np.argsort(a[:,2])]
b[np.argsort(b[:,1], kind='stable')]
```

pandas dataframe

```
df.sort_values(['price', 'weight'])
```

Pandas vs Numpy

Adding columns is way better with Pandas, syntactically and architecturally:

id	price	weight	price per gram
1	10	100	0.1
2	20	80	0.13
3	15	200	0.08
4	15	80	0.19

numpy array

```
np.column_stack((a, a[:,1]/a[:,2]))
```

pandas dataframe

```
df['price_per_gram'] = df.price/df.weight
```

Credits Lev Maximov

47

Pandas vs Numpy

With NumPy arrays, even if the element you search for is the first one, you'll still need time proportional to the size of the array to find it. With Pandas, you can index the column(s) you expect to be queried most often and reduce search time to a constant.

df				df1			
id	price	weight			price	weight	
100	10	100	df.set_index('id') → ← df1.reset_index()	100	10	100	• read-only • unique • pre-heating
314	20	150		314	20	150	
213	15	200		213	15	200	
432	15	80		432	15	80	

numpy array

```
a[np.where(a[0]==213)[0][0]][2]
```

O(N)

pandas dataframe

```
df.loc[df.id==213, 'weight'].item()
```

O(N)

```
df.set_index('id', inplace=True)
```

```
df.loc[213, 'weight']
```

```
df.reset_index(inplace=True)
```

O(1)

Credits Lev Maximov

Pandas vs Numpy

If you want to complement a table with information from another table based on a common column, NumPy is hardly any help. Pandas is better, especially for 1:n relationships.

sales

client product_id

John	1
Silvia	3
Andrew	1

products

id name

1	banana
2	apple
3	orange

➔

client product_id name

John	1	banana
Silvia	3	orange
Andrew	1	banana

numpy array

-

pandas dataframe

```
sales.join(products.set_index('id'),  
            on='product_id')
```

sql

```
select * from sales join products  
on sales.product_id=products.id;
```

Credits Lev Maximov

49

Pandas vs Numpy

One of the most interesting features of Pandas is that of “Pivot” tables. The idea is similar to projecting a multi-dimensional space into a two dimensional plane.

product client	client	product	quantity	➔	product client	bananas	oranges	product client
	John	bananas	5		John	5	3	
	John	oranges	3		Silvia	4	2	
	Silvia	bananas	4					
	Silvia	oranges	2					

numpy array

-

pandas dataframe

```
df.pivot(index='client', columns='product',  
          values='quantity')
```

sql

```
pivot (SQL Server, Oracle)  
model clause (Oracle)  
crosstab function (PostgreSQL)
```

Credits Lev Maximov

50

Performance Tips

Optimize your Pandas code for better performance:

- Use vectorized operations instead of loops
- Use `.loc` and `.iloc` for faster indexing
- Filter data early to reduce the size of your dataset
- Use appropriate data types to reduce memory usage
- Consider using `pandas.read_csv()` with `usecols` to load only needed columns
- For large datasets, consider chunking with `chunksize` parameter
- Use `inplace=True` when appropriate to avoid copies

51

Additional Resources

Official Pandas Documentation:

<https://pandas.pydata.org/docs/>

Pandas Cheat Sheet:

https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

GitHub Repositories:

- Pandas Exercises: https://github.com/guipsamora/pandas_exercises
- 100 Pandas Puzzles: <https://github.com/ajcr/100-pandas-puzzles>

Lev Maximov post on pandas:

<https://medium.com/better-programming/pandas-illustrated-the-definitive-visual-guide-to-pandas-c31fa921a43>

52

Exercises

Load the Iris dataset and perform basic exploratory analysis:

```
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv')
```

- Display the first and last 5 rows of the dataset
- Show the shape and a statistical summary of the numerical columns
- List the unique species in the dataset and count how many samples of each species exist

53

Exercises

Selection and filtering:

- Select only the columns related to petal measurements
- Filter the DataFrame to show only flowers of the 'setosa' species
- Create a new DataFrame containing flowers with sepal length greater than 6 and sepal width less than 3

Grouping and aggregating:

- Group the Iris dataset by species and calculate the mean of each measurement
- Find the minimum and maximum values for each measurement, grouped by species (try to create only one table, `hint* use .agg`)
- Create a new column 'petal_area' by multiplying petal_length and petal_width, then find the average petal area for each species

54

Exercises

Visualization and analysis:

- Create a scatter plot comparing sepal_length and sepal_width, with different colors for each species
- Generate box plots for each measurement, grouped by species
- Identify which species has the largest variance in petal length

SEABORN

Seaborn

Seaborn is a Python data visualization library based on Matplotlib that provides a high-level interface for creating attractive and informative statistical graphics.

Key features:

- Built-in themes for styling Matplotlib graphics
- High-level functions for common statistical visualizations
- Integration with Pandas data structures
- Automatic handling of categorical variables
- Tools for visualizing relationships in complex datasets
- Beautiful default color palettes

57

Installing Seaborn

Seaborn can be installed using pip:

```
pip install seaborn
```

Basic import convention:

```
import seaborn as sns  
import matplotlib.pyplot as plt
```

58

Why use Seaborn?

Seaborn builds on Matplotlib and offers several advantages:

- More aesthetically pleasing default styles
- Built-in support for visualizing relationships in datasets
- Simpler syntax for complex visualizations
- Enhanced support for Pandas DataFrames
- Better handling of categorical variables
- Built-in themes and color palettes
- Statistical estimation and visualization integration

59

Basic plots in Seaborn

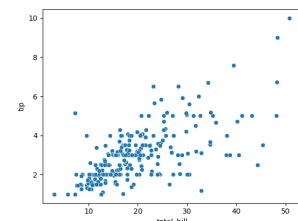
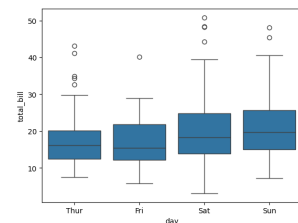
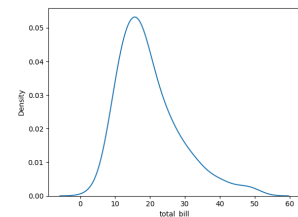
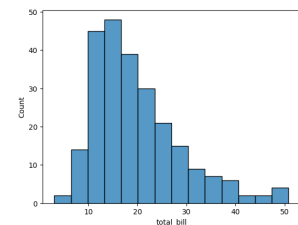
Seaborn provides various plot types, each optimized for different types of data:

```
# Load sample dataset
tips = sns.load_dataset("tips")

# Distribution plots
sns.histplot(tips["total_bill"])
sns.kdeplot(tips["total_bill"])

# Categorical plots
sns.boxplot(x="day", y="total_bill", data=tips)
sns.violinplot(x="day", y="total_bill", data=tips)
sns.barplot(x="day", y="total_bill", data=tips)

# Relationship plots
sns.scatterplot(x="total_bill", y="tip", data=tips)
sns.lineplot(x="total_bill", y="tip", data=tips)
```



Distribution plots

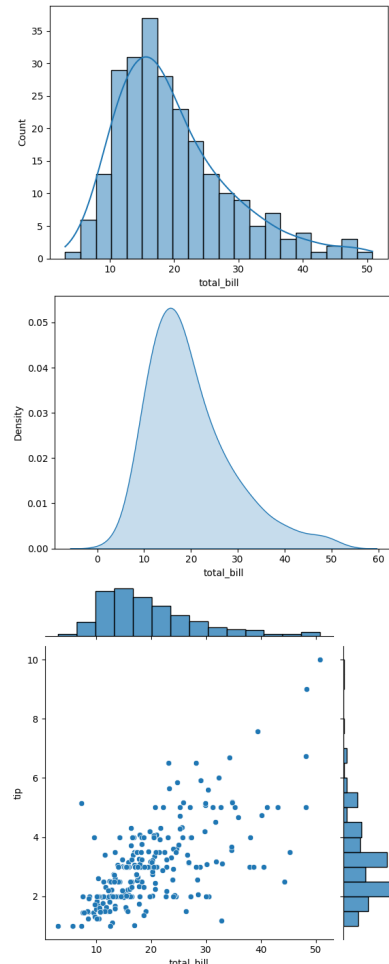
Visualize the distribution of a dataset:

```
# Histogram
sns.histplot(data=tips, x="total_bill", bins=20, kde=True)

# Kernel Density Estimation (KDE)
sns.kdeplot(data=tips, x="total_bill", shade=True)

# Distribution plot
sns.displot(data=tips, x="total_bill", kind="kde")

# Joint distribution of two variables
sns.jointplot(data=tips, x="total_bill", y="tip", kind="scatter")
```



Categorical plots

Visualize the relationship between a categorical variable and a numeric variable:

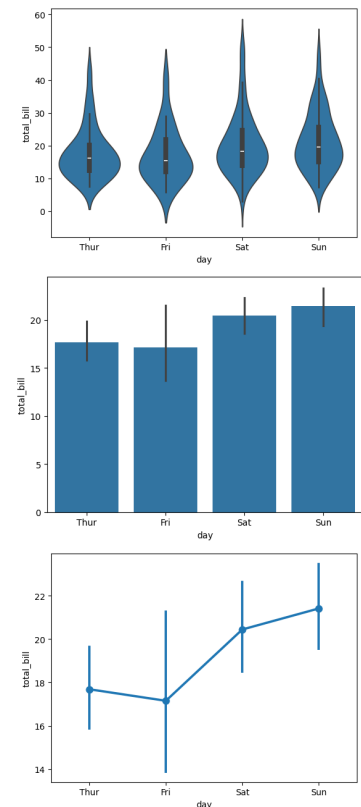
```
# Box plot
sns.boxplot(x="day", y="total_bill", data=tips)

# Violin plot
sns.violinplot(x="day", y="total_bill", data=tips)

# Bar plot (with confidence intervals)
sns.barplot(x="day", y="total_bill", data=tips)

# Point plot (showing mean and confidence interval)
sns.pointplot(x="day", y="total_bill", data=tips)

# Count plot (like a histogram for categorical data)
sns.countplot(x="day", data=tips)
```



Relationship Plots

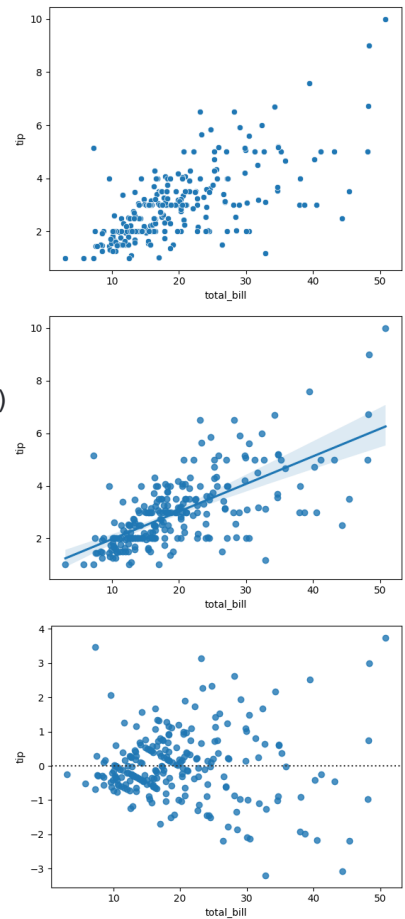
Visualize the relationship between two numeric variables:

```
# Scatter plot
sns.scatterplot(x="total_bill", y="tip", data=tips)

# Line plot
sns.lineplot(x="total_bill", y="tip", data=tips)

# Regression plot (with regression line)
sns.regplot(x="total_bill", y="tip", data=tips)

# Residual plot
sns.residplot(x="total_bill", y="tip", data=tips)
```



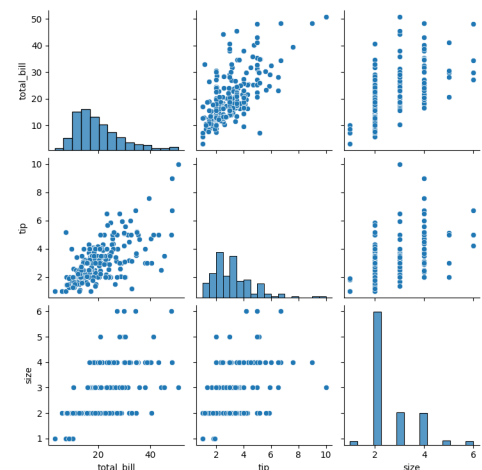
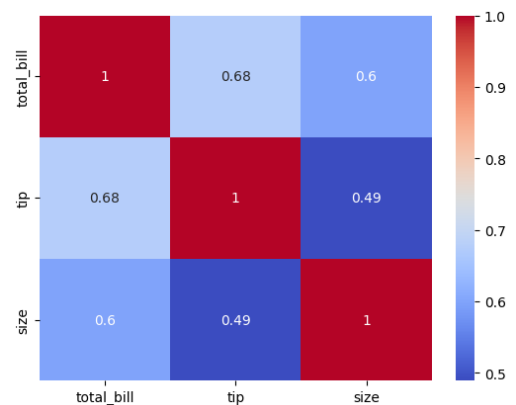
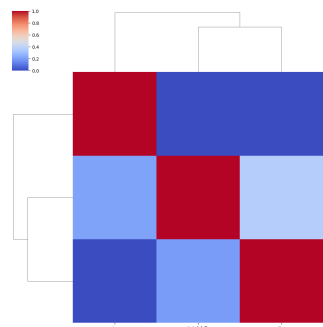
Matrix Plots

Visualize complex datasets with multiple variables:

```
# Correlation heatmap
corr = tips.corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")

# Pairwise relationships
sns.pairplot(tips)

# Cluster map
sns.clustermap(corr, cmap="coolwarm", standard_scale=1)
```



Matrix Plots

```
# example with random matrix
feats = np.random.rand(100,100)
sns.heatmap(feats)

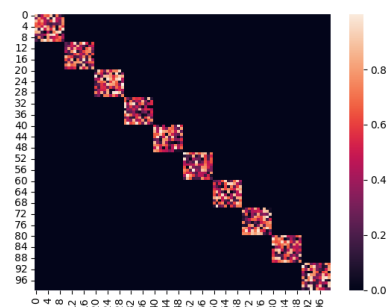
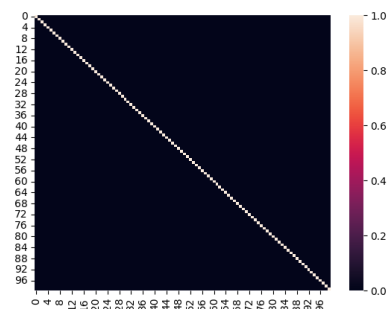
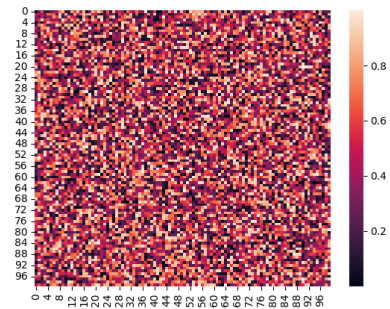
# example with identity matrix
ident = np.identity(100)
sns.heatmap(ident)

# example with diagonal block matrix
block_size = 10
num_blocks = 100 // block_size

# correlated random values
blocks = [np.random.rand(block_size, block_size)
          for _ in range(num_blocks)]

# block diagonal matrix
matrix = block_diag(*blocks)

sns.heatmap(matrix, annot=False)
```



Exercises

Using the Iris dataset:

- Create a pairplot showing relationships between all measurements colored by species
- Generate a heatmap showing the correlation between the four measurements
- Create a violin plot comparing petal length across the three species

SciPy

SciPy

R is a language dedicated to statistics. Python is a general-purpose language with statistics modules. R has more statistical analysis features than Python, and specialized syntaxes.

However, when it comes to building complex analysis pipelines that mix statistics with e.g. image analysis, text mining, or control of a physical experiment, the richness of Python is an invaluable asset

SciPy

File input/output: `scipy.io`

Special functions: `scipy.special`

Linear algebra operations: `scipy.linalg`

Interpolation: `scipy.interpolate`

Optimization and fit: `scipy.optimize`

Statistics and random numbers: `scipy.stats`

Numerical integration: `scipy.integrate`

Fast Fourier transforms: `scipy.fftpack`

Signal processing: `scipy.signal`

Image manipulation: `scipy.ndimage`